

Step 01 – Loading the data

Step 02 – Checking whether any changes are required during data loading

Step 03 – Reviewing all column names and renaming any unusual or invalid column names to usable names. Ex. ALTER TABLE sales\_transaction RENAME COLUMN TransactionID TO TransactionId;

## Data Cleaning

### Sales Transaction table

| <u>Table Name</u>  | <u>Characteristics</u>                                                   |
|--------------------|--------------------------------------------------------------------------|
| TransactionId      | Unique for all transactions                                              |
| CustomerId         | All the customers who purchased, every customer have their own unique id |
| ProductId          | Product Identifier through Id, unique for all unique products            |
| Quantity Purchased | How many quantity of product purchased in that particular transaction    |
| TransactionDate    | Exact date at which the transaction has happened                         |
| Price              | Price per piece for the product                                          |

- Here we have to check for duplicate columns, as TransactionID must be unique for every transaction. If the same TransactionID appears more than once, it should be investigated as a potential duplicate.
- Next step will be to create another table from the previous table but only with distinct values.
  - Checking for duplicate columns

```
• SELECT TransactionID, COUNT(*)  
• FROM sales_transaction  
• GROUP BY TransactionID  
• HAVING COUNT(*) > 1;  
•
```

- Creating new table with only unique transactions and deleting previous table with duplicates

```
• CREATE TABLE sales_transaction_nodupe AS  
• SELECT DISTINCT *  
• FROM sales_transaction;  
•  
• DROP TABLE sales_transaction;  
•  
• ALTER TABLE sales_transaction_nodupe  
• RENAME TO sales_transaction;  
•  
• SELECT * FROM sales_transaction;
```

- The TransactionDate column needs to be converted from the DATE data type to a text format for further processing.
- A new table is created with the updated date column, after which the original table is replaced by the new table.

```

• create table sales_transaction1 as
• select TransactionID, CustomerID, ProductID, QuantityPurchased, TransactionDate, Price,
• cast(TransactionDate as char (50)) TransactionDate_updated
• from sales_transaction;
•
• drop table sales_transaction;
•
• alter table sales_transaction1
• rename to sales_transaction;
•
• select * from sales_transaction;

```

### Product Inventory Table

| <u>Table Name</u> | <u>Characteristics</u>                                                                           |
|-------------------|--------------------------------------------------------------------------------------------------|
| Product_id        | Unique Product id for every product, will be the foreign key for sales transaction table as for. |
| Product_name      | Name of all products , customers can identify with this                                          |
| Category          | Specific category of the product – Clothing, Home And Kitchen, Beauty and Health, Electronics    |
| Stocklevel        | How many available in inventory                                                                  |
| Price             | Price of the particular product                                                                  |

- The primary task is to verify whether the product prices in the inventory table match the prices recorded in the sales transaction table.
- Any mismatched prices in the sales transaction table will be updated using the corresponding inventory price as the reference price.
  - Note – Inventory price will be considered actual price

```

• select t.TransactionID, t.price as TransactionPrice, p.price as InventoryPrice
• from sales_transaction t
• left join product_inventory p
• on p.productID = t.productID
• where t.price != p.price;
•
• update sales_transaction s
• join product_inventory p
• on p.productID = s.productID
• set s.price = p.price
• where s.price != p.price;
• select * from sales_transaction;

```

### Customer Profile Table

| <u>Table</u> | <u>Characteristics</u>                                 |
|--------------|--------------------------------------------------------|
| CustomerId   | Unique Customer id for all customers                   |
| Age          | Age of customers                                       |
| Gender       | Gender of customer                                     |
| Location     | Which location the belong to – Nort, East, South, West |
| Join Date    | At the exact date when customers joined the platform   |

- Here we will check for any null or blank value in any column.
- Will update all null values with 'Unknown'

```
• select count(*) from customer_profiles
• where customerid is Null
• or age is null or gender is null or location is null or joindate
  is null;
•
• update customer_profiles set customerid = "Unknown"
• where customerid is Null;
• update customer_profiles set age = "Unknown"
• where age is null;
• update customer_profiles set gender = "Unknown"
• where gender is null;
• update customer_profiles set location = "Unknown"
• where location is null;
• update customer_profiles set joindate = "Unknown"
• where joindate is null;
•
• select * from customer_profiles;
```

## Data Analysis

- This analysis summarizes the total quantity sold and total revenue generated for each product.

```
select ProductID, sum(quantitypurchased) as TotalUnitsSold, sum(pr  
ice*quantitypurchased) TotalSales from  
Sales_transaction  
group by ProductID  
order by TotalSales desc;
```

- Identifying customers based on the number of transactions..

```
select CustomerID, count(TransactionID) NumberOfTransactions from  
sales_transaction  
group by CustomerID  
order by NumberOfTransactions desc;
```

- Analyzing category-level demand based on total units sold and total sales revenue.

```
select p.Category, sum(s.QuantityPurchased) as TotalUnitsSold,  
sum(s.price*s.QuantityPurchased) TotalSales  
from sales_transaction s  
join product_inventory p  
on s.productID = p.productID  
group by p.Category  
order by sum(s.price*s.QuantityPurchased) desc;
```

- Finding to 10 most selling Products – For increasing more revenue checking most selling products.

```
select ProductID, sum(quantityPurchased*price) as TotalRevenue  
from sales_transaction  
group by ProductID  
order by sum(quantityPurchased*price) desc limit 10;
```

- Identifying the 10 least-selling products based on total units sold to support inventory optimization and stock-clearance decisions.
  - Checking products which were sold at least 1 time

```
select ProductID, sum(QuantityPurchased) as TotalUnitsSold  
from Sales_transaction  
group by ProductID  
having sum(QuantityPurchased) >0  
order by sum(QuantityPurchased) asc limit 10  
;
```

- Finding products which were not sold a single time

```
select ProductID, sum(QuantityPurchased) as TotalUnitsSold
from Sales_transaction
group by ProductID
having sum(QuantityPurchased) = 0
order by sum(QuantityPurchased) asc limit 10
;
```

- Analyzing daily transaction activity to understand transaction volume, units sold, and total sales over time.

```
• select transactiondate_updated as DATETRANS, count(transactionID)
  as Transaction_count,
• sum(quantitypurchased) TotalUnitsSold, round(sum(price*quantitypu
  rchased),2) TotalSales
•
• from sales_transaction
•
• group by transactiondate_updated
• order by transactiondate_updated desc;
•
```

- Analyzing month-over-month sales performance to identify changes and growth trends over time.

```
• with cte as (select
• month(transactiondate_updated) month ,
• round(sum(quantitypurchased*price),2) total_sales
• from sales_transaction
• group by month(transactiondate_updated))
• select month,total_sales,
• lag(total_sales) over (order by month ) previous_month_sales,
• round((total_sales - lag(total_sales) over (order by month ) )*10
  0/
• lag(total_sales) over (order by month ) ,2) mom_growth_percentage
• from cte ;
```

- Analyzing customer-level transaction frequency and total spending to understand purchasing patterns.

```
• select CustomerID, count(transactionID) NumberOfTransactions,
• sum(quantityPurchased*price) TotalSpent
• from sales_transaction
• group by CustomerID
• order by sum(quantityPurchased*price) desc
• ;
```

- Identifying customers who have spent more than 1,000 and completed at least 10 transactions for potential promotional offers.

```
select CustomerID, count(transactionID) NumberOfTransactions,
sum(quantityPurchased*price) TotalSpent
from sales_transaction
group by CustomerID
having sum(quantityPurchased*price)>1000 and count(transactionID)
>10
order by sum(quantityPurchased*price) desc
;
```

- For this analysis, occasional customers are defined as customers with two or fewer transactions.

```
select CustomerID, count(transactionID) NumberOfTransactions,
sum(quantityPurchased*price) TotalSpent
from sales_transaction
group by CustomerID
having count(transactionID)<=2
order by count(transactionID) asc, sum(quantityPurchased*price) desc
;
```

- Finding customer who didn't purchase anything till date

```
SELECT c.CustomerID,
COUNT(s.TransactionID) AS NumberOfTransactions
FROM customer_profiles c
LEFT JOIN sales_transaction s
ON c.CustomerID = s.CustomerID
GROUP BY c.CustomerID
HAVING COUNT(s.TransactionID) = 0;
```

- Customer wise purchase of products – To help get proper information about demography wise distribution of product demand (Note Identifying products that individual customers purchased more than once.)

```
select CustomerID, ProductID,
count(ProductID) TimesPurchased
from sales_transaction
group by CustomerID, ProductID
having count(ProductID) >1
order by count(ProductID) desc;
```

- Identifying the first and most recent purchase dates for each customer and calculating the number of days between them.

```
select CustomerID,min(Transactiondate_updated) FirstPurchase,max(
Transactiondate_updated) LastPurchase,
datediff(max(Transactiondate_updated) , min(Transactiondate_updat
ed) ) DaysBetweenPurchases
from sales_transaction
group by CustomerID
having datediff(max(Transactiondate_updated) , min(Transactiondat
e_updated) ) >0
order by datediff(max(Transactiondate_updated) , min(Transactiond
ate_updated) ) desc;
```

- Segmenting customers based on their total quantity purchased.

```
create table customer_segment as(
with cte as (
select CustomerID, sum(quantitypurchased) total_quantuty from sales_tra
nsaction
group by CustomerID),
cte_segment as (select *,case
when total_quantuty between 1 and 10 then '1-10'
when total_quantuty between 11 and 30 then '11-30'
when total_quantuty > 30 then '>30'
end 'Total Quantity of Products Purchased',
case
when total_quantuty between 1 and 10 then 'Low'
when total_quantuty between 11 and 30 then 'Med'
when total_quantuty > 30 then 'High'
end 'CustomerSegment'
from cte)
select CustomerSegment, count(*) from cte_segment
group by CustomerSegment)
;

select * from customer_segment;
```